Course Title: Cryptography and Network Security Lab	**Course Code**: BCSE309P
Faculty: Prof. Karthika V	**Slot:** L43 + L44
Regno: 23BCE1145	**Name:** S Vishwajith

EX2B – DES

Code:

Sender:

```
import socket


def circular_left_shift(s, n):
    n = n % len(s)
    return s[n:len(s)] + s[0:n]


def hex_to_bin(s):
    res = ""
    mp = {'0': "0000",
        '1': "0001",
        '2': "0010",
        '3': "0011",
        '4': "0100",
        '5': "0101",
        '6': "0110",
        '7': "0111",
        '8': "1000",
        '9': "1001",
        'A': "1010",
        'B': "1011",
        'C': "1100",
        'D': "1101",
```

```
        'E': "1110",
        'F': "1111"}
    for i in s:
        res += mp[i]
    return res

def bin_to_hex(s):
    res = ""
    mp = {"0000": '0',
        "0001": '1',
        "0010": '2',
        "0011": '3',
        "0100": '4',
        "0101": '5',
        "0110": '6',
        "0111": '7',
        "1000": '8',
        "1001": '9',
        "1010": 'A',
        "1011": 'B',
        "1100": 'C',
        "1101": 'D',
        "1110": 'E',
        "1111": 'F'}
    for i in range(0, len(s), 4):
        ch = s[i:i+4]
        res += mp[ch]
    return res

def permute(s, arr):
    res = ""
```

```
    for i in arr:
        res += s[i-1]
    return res


def get_intermediate_keys(key):
    PC1 = [57, 49, 41, 33, 25, 17, 9,
           1, 58, 50, 42, 34, 26, 18,
           10, 2, 59, 51, 43, 35, 27,
           19, 11, 3, 60, 52, 44, 36,
           63, 55, 47, 39, 31, 23, 15,
           7, 62, 54, 46, 38, 30, 22,
           14, 6, 61, 53, 45, 37, 29,
           21, 13, 5, 28, 20, 12, 4]
    PC2 = [14, 17, 11, 24, 1, 5,
           3, 28, 15, 6, 21, 10,
           23, 19, 12, 4, 26, 8,
           16, 7, 27, 20, 13, 2,
           41, 52, 31, 37, 47, 55,
           30, 40, 51, 45, 33, 48,
           44, 49, 39, 56, 34, 53,
           46, 42, 50, 36, 29, 32]
    round_shift = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]

    key_permuted = permute(key, PC1)
    left = key_permuted[:28]
    right = key_permuted[28:]
    key_list = []
    for i in range(len(round_shift)):
        left = circular_left_shift(left, round_shift[i])
        right = circular_left_shift(right, round_shift[i])
        key_list.append(permute(left+right, PC2))
```

```
    return key_list

def exclusive_or(s1, s2):
    res = ""
    for (i, j) in zip(s1, s2):
        res += str(int(i) ^ int(j))
    return res

def S_box(s):
    sbox = [[[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
        [0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
        [4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
        [15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]],

        [[15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
        [3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
        [0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
        [13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9]],

        [[10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
        [13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
        [13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
        [1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12]],

        [[7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],
        [13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9],
        [10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4],
        [3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14]],

        [[2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9],
        [14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6],
```

```
        [4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14],
        [11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3]],

        [[12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11],
        [10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8],
        [9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6],
        [4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13]],

        [[4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1],
        [13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6],
        [1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2],
        [6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12]],

        [[13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7],
        [1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2],
        [7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8],
        [2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11]]]

    res = ""
    for i in range(0, len(s), 6):
        curr_sbox = sbox[i//6]
        curr_s = s[i:i+6]
        row = int(curr_s[0] + curr_s[-1], 2)
        col = int(curr_s[1:-1], 2)
        res += bin(curr_sbox[row][col])[2:].rjust(4, '0')
    return res

def DES(pt, key, decrypt = False):
    IP = [58, 50, 42, 34, 26, 18, 10, 2,
        60, 52, 44, 36, 28, 20, 12, 4,
        62, 54, 46, 38, 30, 22, 14, 6,
```

```
    64, 56, 48, 40, 32, 24, 16, 8,
    57, 49, 41, 33, 25, 17, 9, 1,
    59, 51, 43, 35, 27, 19, 11, 3,
    61, 53, 45, 37, 29, 21, 13, 5,
    63, 55, 47, 39, 31, 23, 15, 7]
EP_D = [32, 1, 2, 3, 4, 5, 4, 5,
        6, 7, 8, 9, 8, 9, 10, 11,
        12, 13, 12, 13, 14, 15, 16, 17,
        16, 17, 18, 19, 20, 21, 20, 21,
        22, 23, 24, 25, 24, 25, 26, 27,
        28, 29, 28, 29, 30, 31, 32, 1]
P32 = [16, 7, 20, 21,
    29, 12, 28, 17,
    1, 15, 23, 26,
    5, 18, 31, 10,
    2, 8, 24, 14,
    32, 27, 3, 9,
    19, 13, 30, 6,
    22, 11, 4, 25]
IP_1 = [40, 8, 48, 16, 56, 24, 64, 32,
        39, 7, 47, 15, 55, 23, 63, 31,
        38, 6, 46, 14, 54, 22, 62, 30,
        37, 5, 45, 13, 53, 21, 61, 29,
        36, 4, 44, 12, 52, 20, 60, 28,
        35, 3, 43, 11, 51, 19, 59, 27,
        34, 2, 42, 10, 50, 18, 58, 26,
        33, 1, 41, 9, 49, 17, 57, 25]

pt = hex_to_bin(pt)
pt = permute(pt, IP)
print("\nAfter initial permutation: " + bin_to_hex(pt))
```

```
    key = hex_to_bin(key)
    keys = get_intermediate_keys(key)
    if decrypt:
        keys.reverse()

    left = pt[:32]
    right = pt[32:]

    for i in range(16):
        right_exp = permute(right, EP_D)
        right_exp = exclusive_or(right_exp, keys[i])
        right_exp = S_box(right_exp)
        right_exp = permute(right_exp, P32)
        right_exp = exclusive_or(right_exp, left)
        left, right = right, right_exp
        print("Round ", i + 1, " ", bin_to_hex(left)," ", bin_to_hex(right), " ", bin_to_hex(keys[i]))

    return permute(right + left, IP_1)

print("S-DES sender initialized.\n")
pt = input("Enter a 64-bit plain text in hexadecimal: ")
key = input("Enter a 64-bit key in hexadecimal: ")

ct = bin_to_hex(DES(pt, key))
print(f"\nThe cipher text is: {ct}.")

s = socket.socket()
s.connect(('127.0.0.1', 5995))
s.send(ct.encode().ljust(1024, b'\0'))
s.send(key.encode().ljust(1024, b'\0'))
```

```
print("\nThe Cipher text and the key have been sent successfully to the receiver.")
s.close()
```

Receiver:

```
import socket
def circular_left_shift(s, n):
    n = n % len(s)
    return s[n:len(s)] + s[0:n]

def hex_to_bin(s):
    res = ""
    mp = {'0': "0000",
        '1': "0001",
        '2': "0010",
        '3': "0011",
        '4': "0100",
        '5': "0101",
        '6': "0110",
        '7': "0111",
        '8': "1000",
        '9': "1001",
        'A': "1010",
        'B': "1011",
        'C': "1100",
        'D': "1101",
        'E': "1110",
        'F': "1111"}
    for i in s:
        res += mp[i]
    return res
```

```
def bin_to_hex(s):
    res = ""
    mp = {"0000": '0',
        "0001": '1',
        "0010": '2',
        "0011": '3',
        "0100": '4',
        "0101": '5',
        "0110": '6',
        "0111": '7',
        "1000": '8',
        "1001": '9',
        "1010": 'A',
        "1011": 'B',
        "1100": 'C',
        "1101": 'D',
        "1110": 'E',
        "1111": 'F'}
    for i in range(0, len(s), 4):
        ch = s[i:i+4]
        res += mp[ch]
    return res

def permute(s, arr):
    res = ""
    for i in arr:
        res += s[i-1]
    return res

def get_intermediate_keys(key):
    PC1 = [57, 49, 41, 33, 25, 17, 9,
```

```
        1, 58, 50, 42, 34, 26, 18,
        10, 2, 59, 51, 43, 35, 27,
        19, 11, 3, 60, 52, 44, 36,
        63, 55, 47, 39, 31, 23, 15,
        7, 62, 54, 46, 38, 30, 22,
        14, 6, 61, 53, 45, 37, 29,
        21, 13, 5, 28, 20, 12, 4]
    PC2 = [14, 17, 11, 24, 1, 5,
        3, 28, 15, 6, 21, 10,
        23, 19, 12, 4, 26, 8,
        16, 7, 27, 20, 13, 2,
        41, 52, 31, 37, 47, 55,
        30, 40, 51, 45, 33, 48,
        44, 49, 39, 56, 34, 53,
        46, 42, 50, 36, 29, 32]
    round_shift = [1, 1, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 1]

    key_permuted = permute(key, PC1)
    left = key_permuted[:28]
    right = key_permuted[28:]
    key_list = []
    for i in range(len(round_shift)):
        left = circular_left_shift(left, round_shift[i])
        right = circular_left_shift(right, round_shift[i])
        key_list.append(permute(left+right, PC2))
    return key_list

def exclusive_or(s1, s2):
    res = ""
    for (i, j) in zip(s1, s2):
        res += str(int(i) ^ int(j))
```

```
    return res

def S_box(s):
    sbox = [[[14, 4, 13, 1, 2, 15, 11, 8, 3, 10, 6, 12, 5, 9, 0, 7],
        [0, 15, 7, 4, 14, 2, 13, 1, 10, 6, 12, 11, 9, 5, 3, 8],
        [4, 1, 14, 8, 13, 6, 2, 11, 15, 12, 9, 7, 3, 10, 5, 0],
        [15, 12, 8, 2, 4, 9, 1, 7, 5, 11, 3, 14, 10, 0, 6, 13]],

        [[15, 1, 8, 14, 6, 11, 3, 4, 9, 7, 2, 13, 12, 0, 5, 10],
        [3, 13, 4, 7, 15, 2, 8, 14, 12, 0, 1, 10, 6, 9, 11, 5],
        [0, 14, 7, 11, 10, 4, 13, 1, 5, 8, 12, 6, 9, 3, 2, 15],
        [13, 8, 10, 1, 3, 15, 4, 2, 11, 6, 7, 12, 0, 5, 14, 9]],

        [[10, 0, 9, 14, 6, 3, 15, 5, 1, 13, 12, 7, 11, 4, 2, 8],
        [13, 7, 0, 9, 3, 4, 6, 10, 2, 8, 5, 14, 12, 11, 15, 1],
        [13, 6, 4, 9, 8, 15, 3, 0, 11, 1, 2, 12, 5, 10, 14, 7],
        [1, 10, 13, 0, 6, 9, 8, 7, 4, 15, 14, 3, 11, 5, 2, 12]],

        [[7, 13, 14, 3, 0, 6, 9, 10, 1, 2, 8, 5, 11, 12, 4, 15],
        [13, 8, 11, 5, 6, 15, 0, 3, 4, 7, 2, 12, 1, 10, 14, 9],
        [10, 6, 9, 0, 12, 11, 7, 13, 15, 1, 3, 14, 5, 2, 8, 4],
        [3, 15, 0, 6, 10, 1, 13, 8, 9, 4, 5, 11, 12, 7, 2, 14]],

        [[2, 12, 4, 1, 7, 10, 11, 6, 8, 5, 3, 15, 13, 0, 14, 9],
        [14, 11, 2, 12, 4, 7, 13, 1, 5, 0, 15, 10, 3, 9, 8, 6],
        [4, 2, 1, 11, 10, 13, 7, 8, 15, 9, 12, 5, 6, 3, 0, 14],
        [11, 8, 12, 7, 1, 14, 2, 13, 6, 15, 0, 9, 10, 4, 5, 3]],

        [[12, 1, 10, 15, 9, 2, 6, 8, 0, 13, 3, 4, 14, 7, 5, 11],
        [10, 15, 4, 2, 7, 12, 9, 5, 6, 1, 13, 14, 0, 11, 3, 8],
        [9, 14, 15, 5, 2, 8, 12, 3, 7, 0, 4, 10, 1, 13, 11, 6],
```

```
        [4, 3, 2, 12, 9, 5, 15, 10, 11, 14, 1, 7, 6, 0, 8, 13]],

        [[4, 11, 2, 14, 15, 0, 8, 13, 3, 12, 9, 7, 5, 10, 6, 1],
        [13, 0, 11, 7, 4, 9, 1, 10, 14, 3, 5, 12, 2, 15, 8, 6],
        [1, 4, 11, 13, 12, 3, 7, 14, 10, 15, 6, 8, 0, 5, 9, 2],
        [6, 11, 13, 8, 1, 4, 10, 7, 9, 5, 0, 15, 14, 2, 3, 12]],

        [[13, 2, 8, 4, 6, 15, 11, 1, 10, 9, 3, 14, 5, 0, 12, 7],
        [1, 15, 13, 8, 10, 3, 7, 4, 12, 5, 6, 11, 0, 14, 9, 2],
        [7, 11, 4, 1, 9, 12, 14, 2, 0, 6, 10, 13, 15, 3, 5, 8],
        [2, 1, 14, 7, 4, 10, 8, 13, 15, 12, 9, 0, 3, 5, 6, 11]]]

    res = ""
    for i in range(0, len(s), 6):
        curr_sbox = sbox[i//6]
        curr_s = s[i:i+6]
        row = int(curr_s[0] + curr_s[-1], 2)
        col = int(curr_s[1:-1], 2)
        res += bin(curr_sbox[row][col])[2:].rjust(4, '0')
    return res

def DES(pt, key, decrypt = False):
    IP = [58, 50, 42, 34, 26, 18, 10, 2,
        60, 52, 44, 36, 28, 20, 12, 4,
        62, 54, 46, 38, 30, 22, 14, 6,
        64, 56, 48, 40, 32, 24, 16, 8,
        57, 49, 41, 33, 25, 17, 9, 1,
        59, 51, 43, 35, 27, 19, 11, 3,
        61, 53, 45, 37, 29, 21, 13, 5,
        63, 55, 47, 39, 31, 23, 15, 7]
    EP_D = [32, 1, 2, 3, 4, 5, 4, 5,
```

```
        6, 7, 8, 9, 8, 9, 10, 11,
        12, 13, 12, 13, 14, 15, 16, 17,
        16, 17, 18, 19, 20, 21, 20, 21,
        22, 23, 24, 25, 24, 25, 26, 27,
        28, 29, 28, 29, 30, 31, 32, 1]
P32 = [16, 7, 20, 21,
    29, 12, 28, 17,
    1, 15, 23, 26,
    5, 18, 31, 10,
    2, 8, 24, 14,
    32, 27, 3, 9,
    19, 13, 30, 6,
    22, 11, 4, 25]
IP_1 = [40, 8, 48, 16, 56, 24, 64, 32,
        39, 7, 47, 15, 55, 23, 63, 31,
        38, 6, 46, 14, 54, 22, 62, 30,
        37, 5, 45, 13, 53, 21, 61, 29,
        36, 4, 44, 12, 52, 20, 60, 28,
        35, 3, 43, 11, 51, 19, 59, 27,
        34, 2, 42, 10, 50, 18, 58, 26,
        33, 1, 41, 9, 49, 17, 57, 25]

pt = hex_to_bin(pt)
pt = permute(pt, IP)
print("\nAfter initial permutation: " + bin_to_hex(pt))

key = hex_to_bin(key)
keys = get_intermediate_keys(key)
if decrypt:
    keys.reverse()
```

```
    left = pt[:32]
    right = pt[32:]

    for i in range(16):
        right_exp = permute(right, EP_D)
        right_exp = exclusive_or(right_exp, keys[i])
        right_exp = S_box(right_exp)
        right_exp = permute(right_exp, P32)
        right_exp = exclusive_or(right_exp, left)
        left, right = right, right_exp
        print("Round ", i + 1, " ", bin_to_hex(left)," ", bin_to_hex(right), " ", bin_to_hex(keys[i]))

    return permute(right + left, IP_1)

server = socket.socket()
server.bind(('127.0.0.1', 5995))
server.listen(1)
print("DES Receiver initialized and listening for sender.")
conn, addr = server.accept()
print(f"Connection established with sender at {addr}.")

ct = conn.recv(1024).decode().rstrip('\0')
print(f"\nThe received cipher text is: {ct}.")
key = conn.recv(1024).decode().rstrip('\0')
conn.close()
server.close()

pt = DES(ct, key, decrypt=True)
print(f"\nThe plain text is: {bin_to_hex(pt)}.")
```

Output:

Sender:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-5 (DES)> python .\sender.py
S-DES sender initialized.

Enter a 64-bit plain text in hexadecimal: 123456ABCD132536
Enter a 64-bit key in hexadecimal: AABB09182736CCDD

After initial permutation: 14A7D67818CA18AD
Round  1   18CA18AD   5A78E394   194CD072DE8C
Round  2   5A78E394   4A1210F6   4568581ABCCE
Round  3   4A1210F6   B8089591   06EDA4ACF5B5
Round  4   B8089591   236779C2   DA2D032B6EE3
Round  5   236779C2   A15A4B87   69A629FEC913
Round  6   A15A4B87   2E8F9C65   C1948E87475E
Round  7   2E8F9C65   A9FC20A3   708AD2DDB3C0
Round  8   A9FC20A3   308BEE97   34F822F0C66D
Round  9   308BEE97   10AF9D37   84BB4473DCCC
Round  10   10AF9D37   6CA6CB20   02765708B5BF
Round  11   6CA6CB20   FF3C485F   6D5560AF7CA5
Round  12   FF3C485F   22A5963B   C2C1E96A4BF3
Round  13   22A5963B   387CCDAA   99C31397C91F
Round  14   387CCDAA   BD2DD2AB   251B8BC717D0
Round  15   BD2DD2AB   CF26B472   3330C5D9A36D
Round  16   CF26B472   19BA9212   181C5D75C66D

The cipher text is: C0B7A8D05F3A829C.

The Cipher text and the key have been sent successfully to the receiver.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-5 (DES)>
```

Receiver:

```
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-5 (DES)> python .\receiver.py
DES Receiver initialized and listening for sender.
Connection established with sender at ('127.0.0.1', 52712).

The received cipher text is: C0B7A8D05F3A829C.

After initial permutation: 19BA9212CF26B472
Round  1   CF26B472   BD2DD2AB   181C5D75C66D
Round  2   BD2DD2AB   387CCDAA   3330C5D9A36D
Round  3   387CCDAA   22A5963B   251B8BC717D0
Round  4   22A5963B   FF3C485F   99C31397C91F
Round  5   FF3C485F   6CA6CB20   C2C1E96A4BF3
Round  6   6CA6CB20   10AF9D37   6D5560AF7CA5
Round  7   10AF9D37   308BEE97   02765708B5BF
Round  8   308BEE97   A9FC20A3   84BB4473DCCC
Round  9   A9FC20A3   2E8F9C65   34F822F0C66D
Round  10   2E8F9C65   A15A4B87   708AD2DDB3C0
Round  11   A15A4B87   236779C2   C1948E87475E
Round  12   236779C2   B8089591   69A629FEC913
Round  13   B8089591   4A1210F6   DA2D032B6EE3
Round  14   4A1210F6   5A78E394   06EDA4ACF5B5
Round  15   5A78E394   18CA18AD   4568581ABCCE
Round  16   18CA18AD   14A7D678   194CD072DE8C

The plain text is: 123456ABCD132536.
PS C:\Users\vishw\Coding\Cryptography-Sem-6\Week-5 (DES)>
```